

PatternHunter: faster and more sensitive homology search

Bin Ma, John Tromp and Ming Li

Bioinformatics, Vol.18, No.3, pp. 440-445, 2002

Abstract

Motivation: Genomics and proteomics studies routinely depend on homology searches based on the strategy of finding short seed matches which are then extended. The exploding genomic data growth presents a dilemma for DNA homology search techniques: increasing seed size decreases sensitivity whereas decreasing seed size slows down computation.

Results: We present a new homology search algorithm 'PatternHunter' that uses novel seed model for increased sensitivity and new hit-processing techniques for significantly increased speed. At Blast levels of sensitivity, PatternHunter is able to find homologies between sequences as large as human chromosomes, in mere hours on a desktop.

Availability: PatternHunter is available at <http://www.bioinformaticssolutions.com>, as a commercial package. It runs on all platforms that support Java. PatternHunter technology is being patented; commercial use requires a license from BSI, while non-commercial use will be free.

Contact: mli@cs.ucsb.edu

Problem Definition

- Smith-Waterman alignment which compares all bases against all bases is clearly too slow.
- Two lines of approach lead to improvements.
 - ▶ finds short exact 'seed' matches (hits), which are then extended into longer alignments.
 - ▶ uses suffix trees.
- In this paper, we introduce novel seeding schemes and hit-processing method.

Blast (1)

Dictionary:

All words of length k (~ 11)

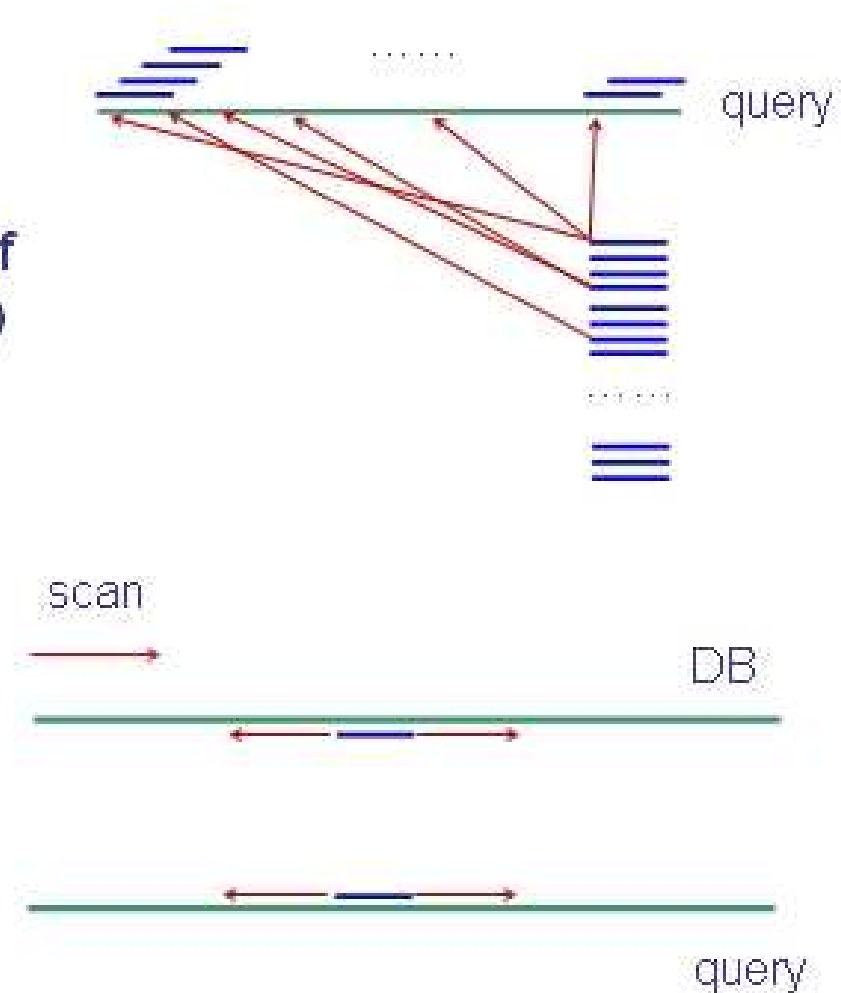
Alignment initiated between words of alignment score $\geq T$ (typically $T = k$)

Alignment:

Ungapped extensions until score below statistical threshold

Output:

All local alignments with score $>$ statistical threshold



Blast (2)

Example:

$k = 4$,

$T = 4$

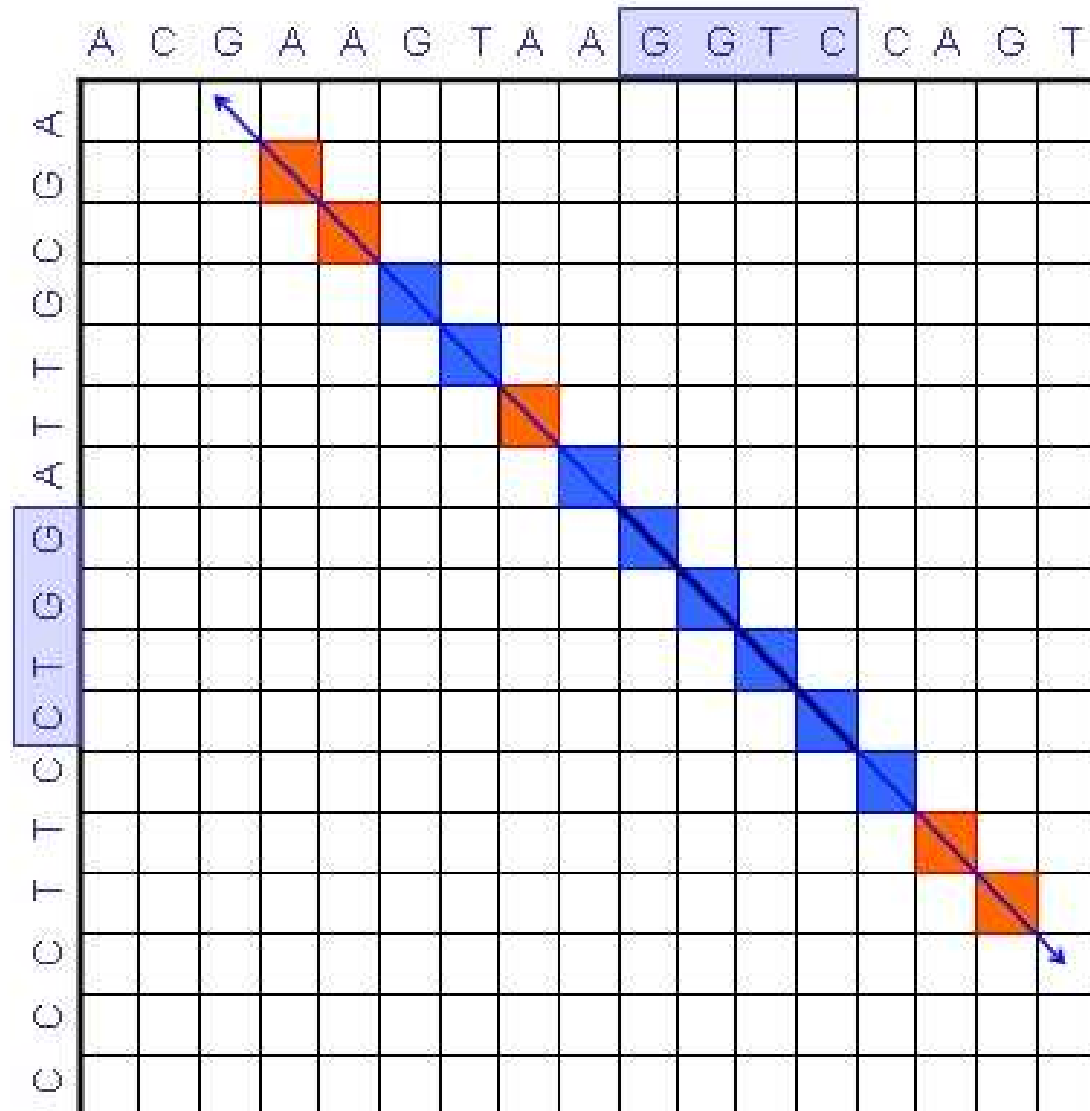
The matching word
GGTC initiates an
alignment

Extension to the left and
right with no gaps until
alignment falls $< 50\%$

Output:

GTAAGGTCC

GTTAGGTCC



PatternHunter (1)

- Blast looks for matches of k (Default $k = 11$ in Blastn) consecutive letters as seeds. Instead we propose to use *nonconsecutive* k letters as seeds. We call relative positions of the k letters a *model*, and k its *weight*.
- We denote a model by a 0-1 string, where the 1-positions represent require matches, while the 0s are 'don't care'.

PatternHunter (2)

■ Example:

- ▶ A weight six model 1110111.

A seed match:

a	c	t	g	a	c	t
a	c	t	t	a	c	t

- ▶ Blast uses models of the form 1^k .

A weight seven model 1111111.

A seed match:

a	c	t	g	a	c	t
a	c	t	g	a	c	t

Selecting Good Seeds

- For weight 11, the most sensitive model is 111010010100110111.
- Generally, fewer bases shared between seed and shifted copies, high sensitivity.

1 1 1 1 0 1 0 0 1 0 1 0 0 1 1 1 0 1 1 1 1

- $weight = 11, length = 18$.
shifted copy shares 5 bases

Result (1)

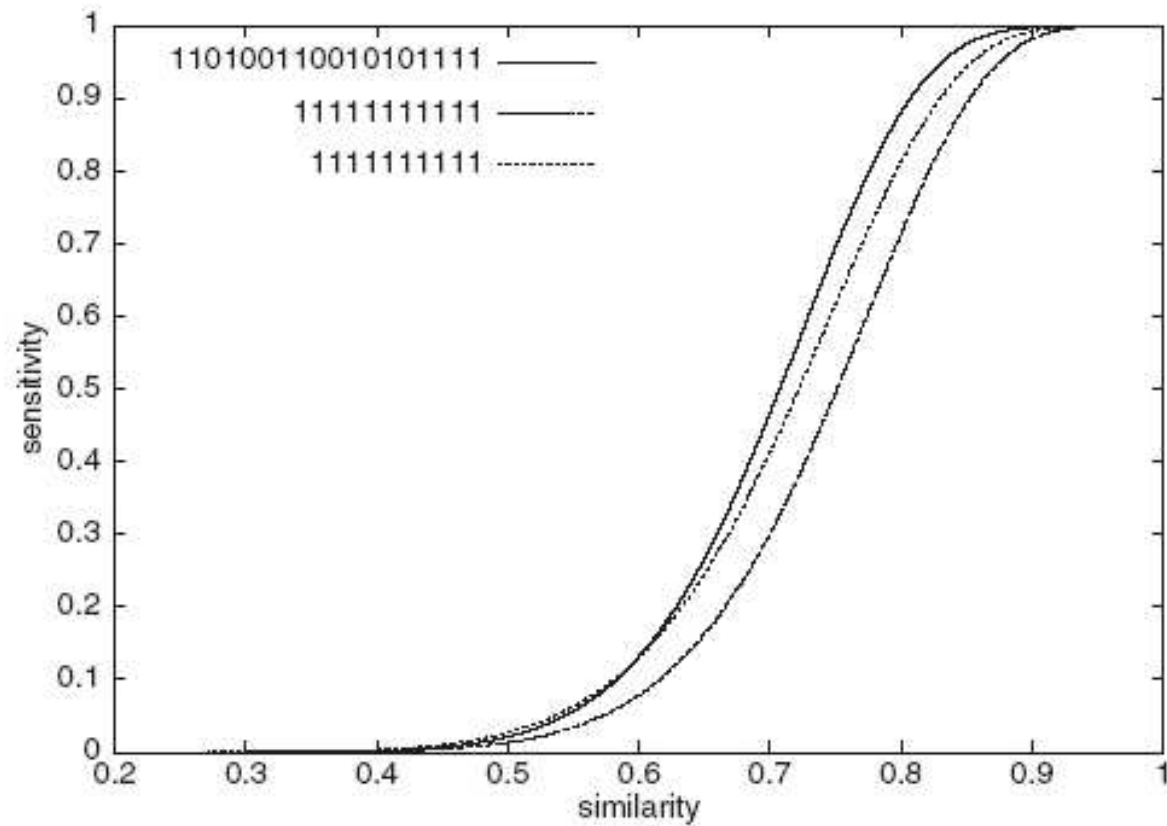


Fig. 1. 1-hit performance of weight 11 spaced model versus weight 11 and 10 consecutive models, coordinates in logarithmic scale.

Result (2)

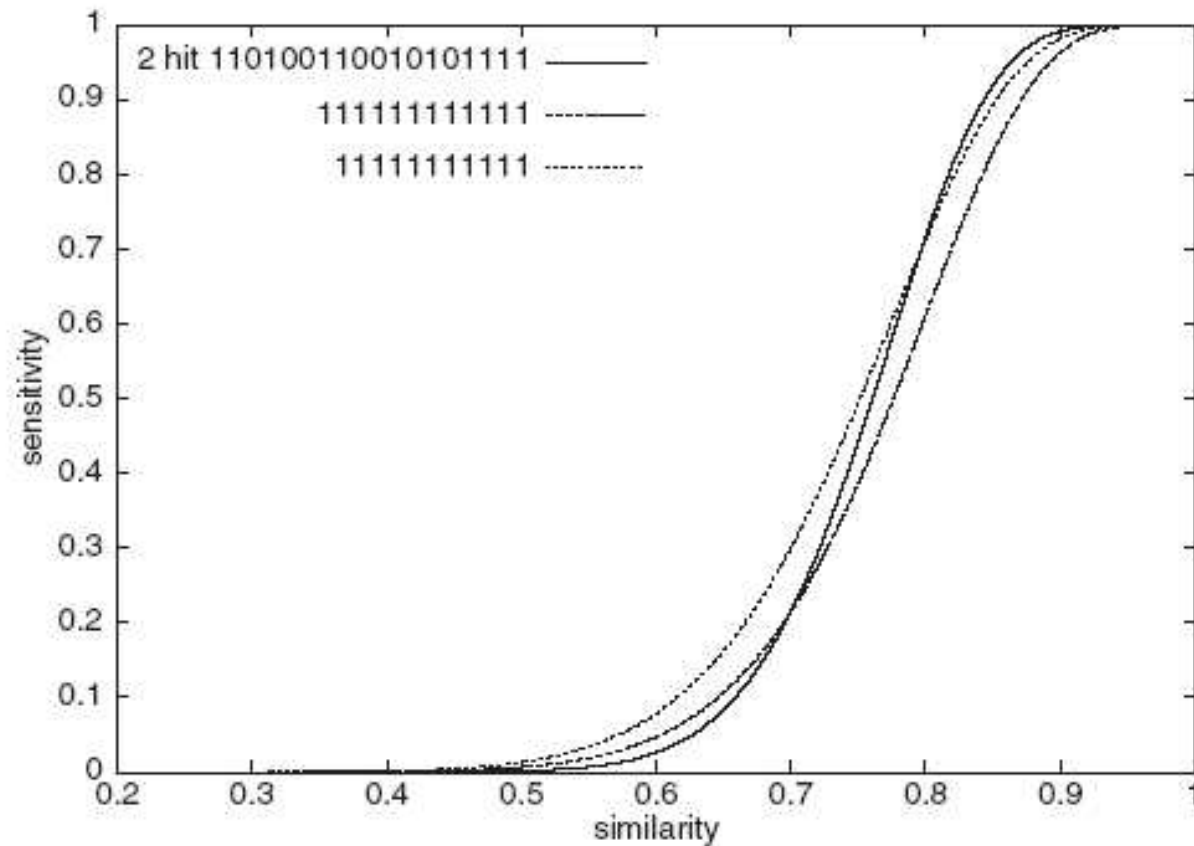


Fig. 2. 2-hit performance of weight 11 spaced model versus single hit weight 11 and 12 consecutive models.

Result (3)

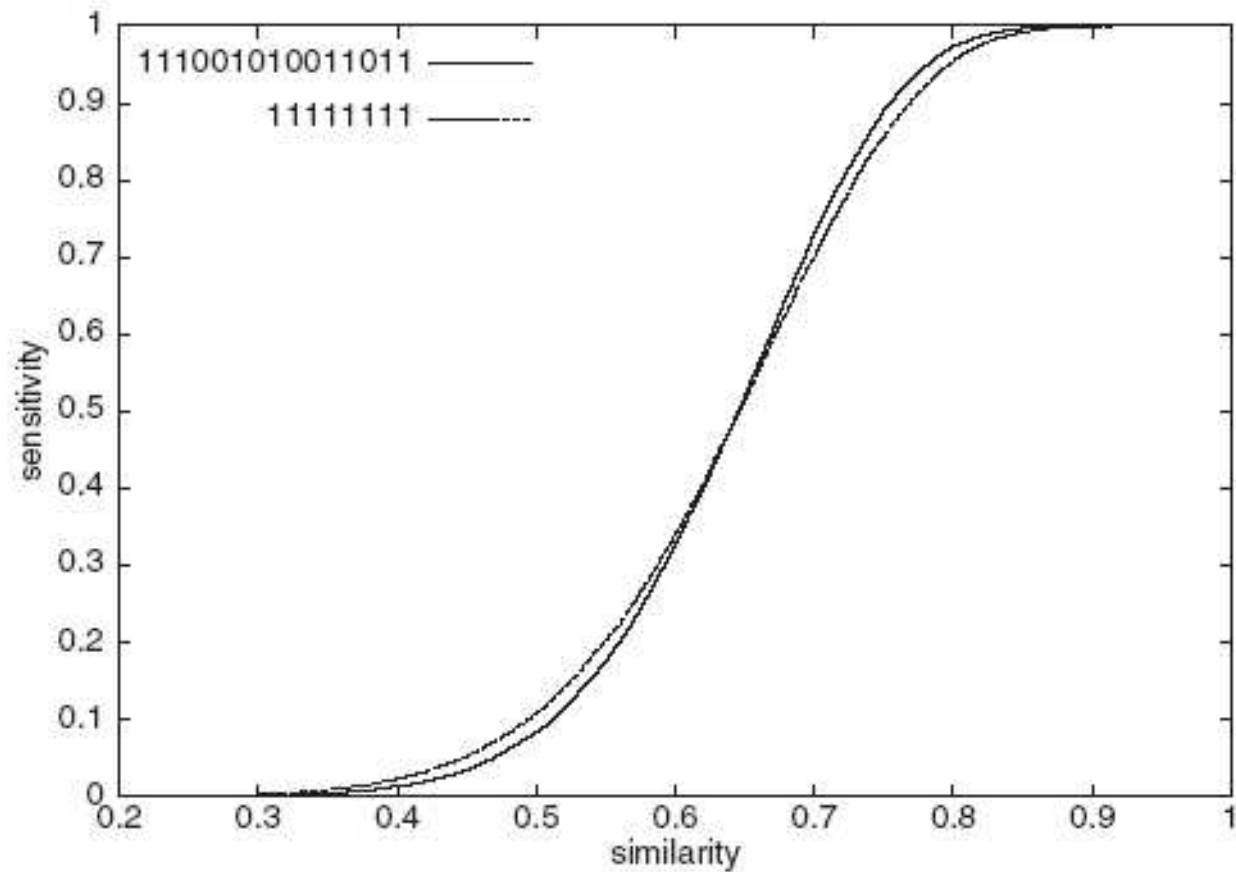


Fig. 3. 1-hit performance of weight 8 consecutive model versus weight 9 nonconsecutive model.

Thanks

Thanks

Date: Sep. 30, 2004

Created by: Liang-Tai Wei